TASK

Django – eCommerce Application Part 2

Visit our website

Introduction

WELCOME TO THE DJANGO E-COMMERCE APPLICATION TASK PART 2!

Now that we have built our eCommerce site, a secure web application with authentication and authorisation, it's time to expand our application to support an API. We will be using our knowledge of authentication and authorisation to build a secure web API for our users. In this lesson, you'll practice both GET and POST requests. The Django REST framework examples show how to create and add data with POST, while the Reddit example focuses on fetching and displaying data with GET. Together, these cover the full cycle of working with APIs.

APIS

You might have noticed that it's possible to link certain websites with your social media accounts. Have you ever wondered how this is possible? Web services can choose to build a web API for their service, making it possible for users to add, remove, update, and delete data from the service. Most social media services have a web API that you can integrate with your service. You can also build your own web API that your users can use to integrate with their services.

In this task, we will take a look at how we can create our own web API for our web service, as well as how we can integrate other APIs with our eCommerce service. We will also learn about Postman and how it can be utilised to test your web API.

Web APIs

API is an acronym for "application programming interface" and it's a set of rules that enable software applications to communicate with each other. Web APIs are APIs that allow applications to communicate over the Internet.

Web APIs use standard communication protocols such as HTTP and HTTPS to allow communication over the Internet. Clients can make a request to the API by sending an HTTP request. The API will then process the request and send back a response. This is similar to how our web service already works, but instead of returning HTML, a web API will typically respond with data in XML (Extensible Markup Language) or JSON (JavaScript Object Notation) format. XML defines a set of rules for encoding documents in a format that is both human- and machine-readable, and JSON is an open standard file format that uses

human-readable text to transmit data objects consisting of attribute-value pairs and array data types.

When building a web API, it's wise to follow some sort of structure or architecture. Using a known standard makes it easier for other developers to use and integrate your API with their projects. It can also assist in preventing issues that can occur later by enabling scalability, flexibility, and independence. We will now take a look at the REST architecture and how we can use it to build a web API for our eCommerce service.

RESTful architecture

In the first task of this two-part task, we asked you to research RESTful APIs. Let's recap some of what you may have learned. REST (representational state transfer) is a software architecture or guideline, invented by Roy Fielding in 2000, for building APIs. The architecture comprises principles and constraints to achieve better performance, scalability, and modifiability.

The main feature of REST is **statelessness**. This means that each request will be treated as a new request and servers do not save client data between requests. Each request has to contain all the information, such as authentication data, a `GET`/`POST`/`PUT`/`DELETE` command, JSON, XML, etc., needed to understand and process the request. Without all the required information the API will not be able to successfully process the request and will ultimately fail to return the desired resource.

Here are some other features and constraints of REST:

Client-server architecture: REST uses the client-server architecture where the client and server communicate over a uniform interface using HTTP.
Uniform interface: RESTful systems have a uniform and consistent interface, which allows simplicity and scalability. We can create a uniform REST interface by following these four constraints:

- **Identification of resources:** Resources are identified with URIs (Uniform Resource Identifiers).
- **Manipulation of resources through representations:** Clients manipulate resources through the representation provided by the server. Representations should provide all information to modify or delete the resource or fetch additional data.
- **Hypermedia as the engine of application state (HATEOAS):** The client doesn't need prior knowledge of the structure of the application's APIs but

can navigate and interact with the application based on the links provided by the server.

Cacheable: When a user makes the same call twice the data is retrieved from the cache instead of making a new API call.

Layered: The client does not know whether it is communicating with an end server or an intermediary such as a gateway.

REPRESENTATIONAL FORMATS

JSON

JSON (JavaScript Object Notation) is a storing and exchanging format that is very popular with web applications. While JSON has "JavaScript" in its name, it's important to note that **the format is language-independent** and only the notation was borrowed from JavaScript.

JSON is human-readable and easy to understand, and it uses key-value pairs similar to a Python dictionary. Values can be of any data type, including string, boolean, integer, float, etc. JSON is also very popular as it can be used in many programming languages, including Python, Java, C#, PHP, and many more.

Here is an example of the JSON format:

```
{
  "name": "Peter Jackson",
  "age": 25,
  "gender": "male",
  "hobbies": ["reading", "gaming", "coding"]
}
```

Take Note

To learn more about JSON you can have a look at the **official website**.

XML

Another format that is also popular to use when communicating data structures is XML (Extensible Markup Language). Just like JSON, XML is platform-independent and human-readable. XML uses tags to mark up elements, and we can create these tags using angle brackets, "<" and ">", as with HTML. Each tag has an opening

and closing component, where the closing tag will have a “/” before the element name in the tag (once again, as with HTML).

Here is an example of a basic element:

```
<person>
    <name>Peter Jackson</name>
    <age>25</age>
</person>
```

We can add attributes to elements as well to provide additional information:

```
<book title="The Catcher in the Rye" author="J.D. Salinger"/>
```

XML documents have a hierarchical structure, which means elements can contain other elements. Here is an example of this:

```
<library>
    <book>
        <title>1984</title>
        <author>George Orwell</author>
    </book>
    <book>
        <title>Brave New World</title>
        <author>Aldous Huxley</author>
    </book>
</library>
```

XML's simplicity and flexibility make it suitable for a variety of applications when exchanging and storing information.

MAKING AN API

Let's see how we can build an API for our web app using the RESTful architecture principles.

Basic API request

We can set up a basic API response to see how they are similar to and differ from the regular HTTP responses you might already be familiar with.

```
def basic_api_response(request):
    if request.method == "GET":
        data = serializers.serialize('json', Store.objects.all())
        return JsonResponse(data=data, safe=False)
```

We first make sure that the user has made a `GET` request. We then run the data from our Store models through a serialiser to serialise it to JSON format. Once we have our data in JSON we can return a **`JsonResponse`** instead of an `HttpResponse`. The data from the response is set to the data produced by our serialiser, and we set the `safe` parameter to `False` to make sure any object instance is allowed to be serialised; without this, we can only serialise dictionary objects.

Output:

```
[{\"model\": \"eCommerce.store\", \"pk\": 1, \"fields\": {\"vendor\": 1,
\"name\": \"BaconShop\", \"description\": \"We sell Bacon!\"}},
{\"model\": \"eCommerce.store\", \"pk\": 2, \"fields\": {\"vendor\": 1,
\"name\": \"Sock & Sandals\", \"description\": \"We sell sock! We sell
sandals! Come check them out!\"}}
```

Take Note

Serialisation in Django is where we translate our objects into different formats. To learn more about this, read the **Django documentation**.

Using the Django REST framework

Unfortunately, the response from the method illustrated above does not produce a very user-friendly resource. When people use our API, we want the experience to be easy and efficient. To achieve this, we can use the Django REST framework to make the serialisation of our data a bit easier. You can install the Django Rest framework using the following command:

```
pip install djangorestframework
```

Sometimes you need a class containing a few attributes that would only be useful to one other class. We can call these classes “helper classes”. These helper classes also have access to the private attributes of the outer class. To create a helper class, we simply nest a class definition within the class we would like to attach the helper class to.

DRF serializers should be in a separate file, `serializers.py`

In `yourApp/serializers.py`

```
from rest_framework import serializers
From .models import Store
class StoreSerializer(serializers.ModelSerializer):
    class Meta:
        model = Store
        fields = ['vendor', 'name', 'description']
```

Here we created a class called `StoreSerializer` (note that Django uses the American spelling for the command, i.e., a “z” rather than an “s”, and so we’ve reflected this in the class name as well; this is not mandatory, but simply provides consistency with the spelling of the command). `StoreSerializer` will be used to serialise the data from our Store model. We set the superclass to `serializers.ModelSerializer`, then immediately create a new class inside our `StoreSerializer` class called `Meta`. This class will reference the model we are looking to serialise and we can also set the fields that should be added during serialisation.

Now let's update our previous view to use our new serialiser:

In our `yourApp/views.py`

```
from django.http import JsonResponse
from .models import Store
From .serializers import StoreSerializer

def view_stores(request):
    if request.method == "GET":
        stores = Stores.objects.all()

        serializer = StoreSerializer(stores, many=True)
        return JsonResponse(data=serializer.data, safe=False)
```

This time we use `StoreSerializer` instead of `serializers.serialize`. We can then make a query inside our serialiser to get all the values from our store table and serialise them. We also have to state in the serialiser that we are expecting multiple values in the query set by setting the `many` parameter to `True`. Finally, we return a `JsonResponse` object with our dataset to the data produced by our serialiser, and again we set the `safe` parameter to `False` to allow the serialisation of all object instances.

After using our new serialiser, our output looks much better:

```
[{"vendor": 1, "name": "BaconShop", "description": "We sell Bacon!"},
{"vendor": 1, "name": "Sock & Sandals", "description": "We sell sock! We
sell sandals! Come check them out!"}]
```

When creating new views, remember to set their endpoints in the **urls.py** folder of your web app.

```
urlpatterns = [
    path('basic_response/', basic_api_response),
    path('get/stores', view_stores),
]
```

We can use the `api_view` decorator from the Django REST framework to make sure we receive `request` instances in our view, as well as to add context to our `Response` objects to allow content negotiation.

Let's see how we can add this to our `view_stores` view:

```
from rest_framework.decorators import api_view

@api_view(['GET'])
def view_stores(request):
    stores = Store.objects.all()
    serializer = StoreSerializer(stores, many=True)
    return Response(serializer.data)
```

We can also have our response return the data in XML format. But first, we need to install a package to assist us:

```
pip install djangorestframework-xml
```

The `djangorestframework-xml` package provides us with serialisers to format our data to XML. First, we have to add the following code to our **settings.py** file:

```
REST_FRAMEWORK = {
    'DEFAULT_RENDERER_CLASSES': (
        'rest_framework_xml.renderers.XMLRenderer',
    )
}
```

This will add the XML renderer to the rendered classes of our project.

We can then import the `render_classes` decorator and apply it to our view using the `XMLRenderer`:

```
from rest_framework.decorators import api_view, renderer_classes
from rest_framework_xml.renderers import XMLRenderer
from rest_framework.response import Response

@api_view(['GET'])
@renderer_classes((XMLRenderer,))
def view_stores(request):
    serializer = StoreSerializer(Store.objects.all(), many=True)
    return Response(serializer.data)
```

The renderer_classes decorator forces this view to render only XML, regardless of any global settings. You will also notice that we return a **`Response`** object instead of a **`JsonResponse`** object. The DRF (djangorestframework) will automatically serialise the response using the defined renderer (in this case, XML)

In addition to enabling users to view data using our API, we can allow them to add data, such as creating and editing stores. We can set up the following view for a user to add a store:

```
from rest_framework import status

@api_view(['POST'])
def add_store(request):

    serializer = StoreSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

We first check to see if the request coming in is a **`POST`** request. We then grab the data from our **`request`** object and run it through our **`StoreSerializer`**. Then we test to see if the data from the request is valid and in the correct format. If it's valid, we save the data and return it to the user with a success response code. Otherwise, we return the error messages with a bad request response code. We can return response codes to users by importing the **`status`** module from **`rest_framework`**.

When allowing users to view and especially add data to and from our database, we need to perform some authentication. Let's look at how to achieve this:

```
@api_view(['POST'])
@authentication_classes([BasicAuthentication])
@permission_classes([IsAuthenticated])
def add_store(request):
    vendor_id = request.data.get('vendor')
    if vendor_id is None:
        return Response({'error': 'Vendor field is required.'},
status=status.HTTP_400_BAD_REQUEST)
    if int(vendor_id) != request.user.id:
        return Response({'error': 'User ID and vendor ID do not match.'},
status=status.HTTP_403_FORBIDDEN)

    serializer = StoreSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

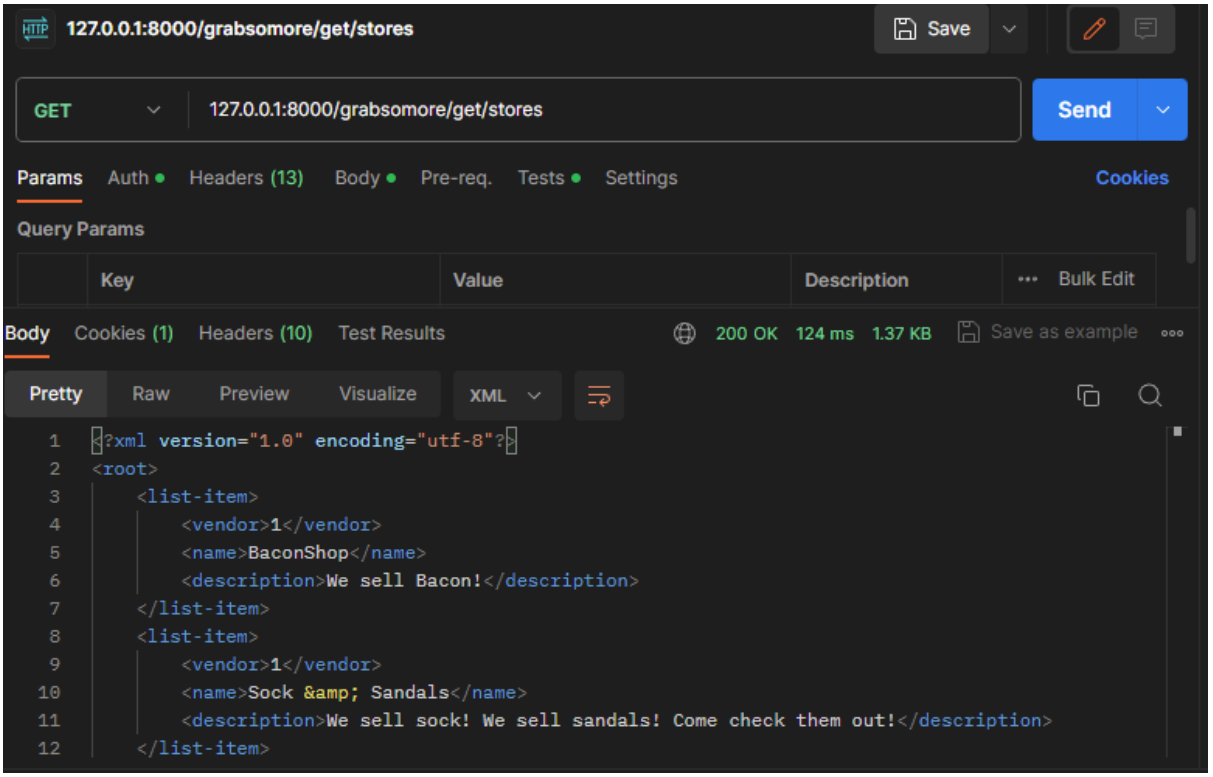
We have now added two more decorators to our function. The first, **`authentication_classes`**, allows us to use a specific authentication method. In this case, we will use **`BasicAuthentication`**, which requires a valid username and password to allow access to the view. We can then determine whether the authentication was successful by using the **`permission_classes`** decorator, adding the **`IsAuthenticated`** permission class to determine whether the user has been authenticated. A bonus of **`BasicAuthentication`** is that it attaches the user to the request. This allows us to determine whether the user is trying to add a new store to their own profile.

POSTMAN

To test our API, it would be quite inefficient to use a browser or to build a new application to make API requests. We can use an application called **Postman** to help us make HTTP requests very easily and attach all the necessary data with it.

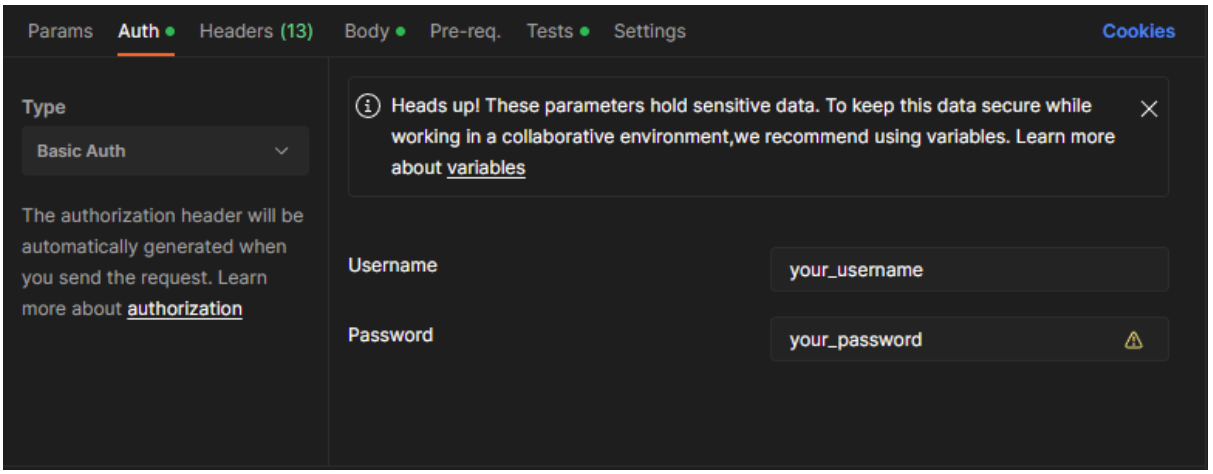
In Postman we can add the endpoints to our API where it says "Enter URL or paste text". We can then set the request method and add the required data.

Let's see what Postman will look like if we make a call to our **get/stores** endpoint:

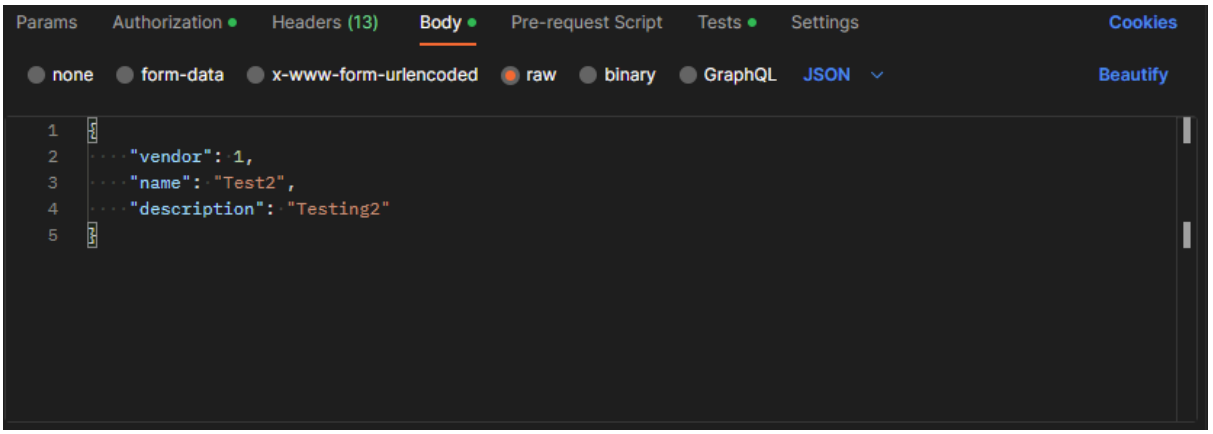


We make a request to the **get/stores** endpoint and set our request method to `GET`, as you can see to the left of the endpoint URI. When we hit send, we get the response you can see in the "Body" at the bottom of the image. This time the API returns the resource in XML form.

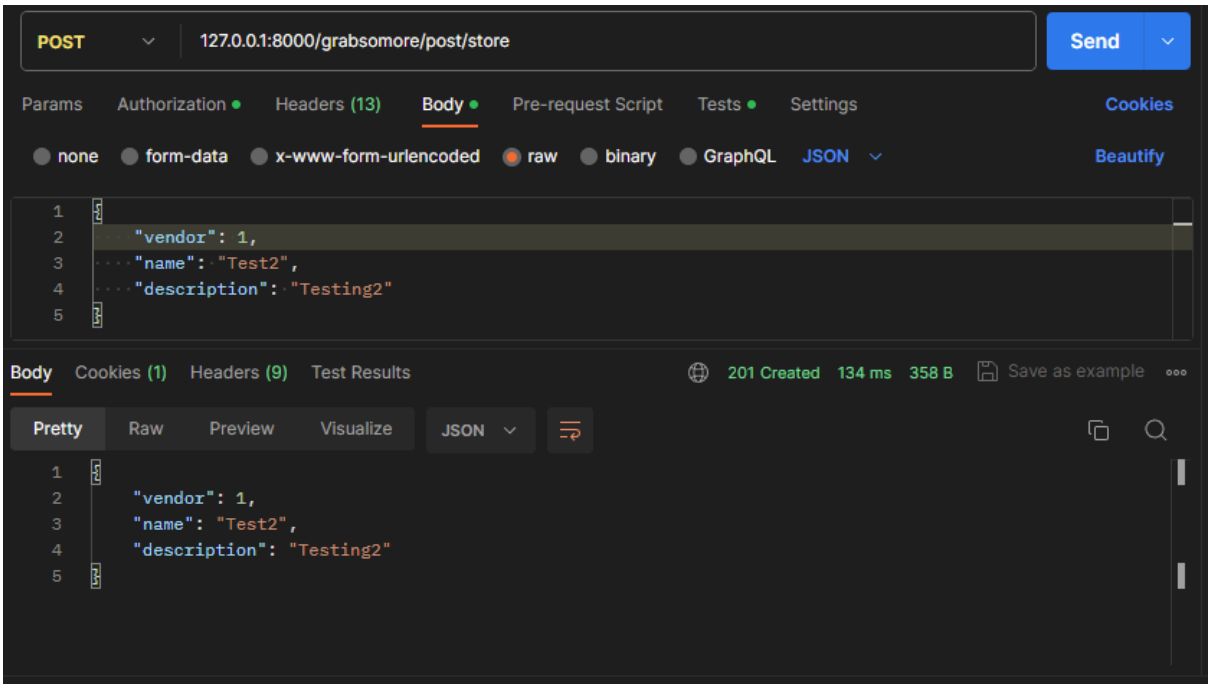
When making a request to an endpoint that requires authentication, you can navigate to the authentication ("Auth") tab below the URL bar. Once on the authentication tab, you can select the type of authentication. We used "Basic Auth" for our **post/store** endpoint.



The **post/store** endpoint also requires the information of a new store in order to add it, and we can add this data to the body of our request. Navigate to the "Body" tab, select the data as "raw", and set the type to "JSON". Then create a store object in JSON format that represents a store in the same way it's produced by our serialiser:



With all our data in place, we can now make a `POST` request to the **post/store** endpoint:



INCORPORATING THIRD-PARTY APIS

`Requests` module

Now that we know how an API works, it becomes possible to incorporate a third-party API into our application. We will use the **requests** module in Python to make API calls. This module allows us to send `HTTP` requests (`GET`, `POST`, `PUT`, `DELETE`, etc.) and handle responses easily.

First, we need a way to make API calls easily within Python, and the **requests** module `enables` us to do this in a simple and efficient manner. With requests, we can send HTTP requests such as `GET`, `POST`, `PUT`, and `DELETE`, and then handle the responses directly in our code. This makes it possible to connect our Django application to external services and retrieve or send data in real time.

Unlike some APIs that require complex authentication steps, Reddit's public JSON endpoints can be accessed with just a `GET` request and a custom **User-Agent** header. This means we can focus on learning how to make requests and parse JSON responses without needing API keys or OAuth setup.

Inside your virtual environment, run the following command in your console to install the **requests** library:

```
pip install requests
```

Now that we have downloaded the module, we can use it by importing it to our Python file:

```
import requests
```

With **`requests`** imported, we can simply use the **`get()`**, **`post()`**, **`put()`**, **`delete()`**, and many more functions available to make our calls. We store the response to the call in a variable as it will contain the resource we have requested.

Here is an example of making a **`GET`** request:

```
r = requests.get('https://api.github.com/events')
```

If we want to add parameters to our **`GET`** request, we can do the following:

```
payload = {'key1': 'value1', 'key2': 'value2'}
r = requests.get('https://api.github.com/events', params=payload)
```

The request URL will look like this:
`'https://api.github.com/events?key1=value1&key2=value2&key2=value3'`

If we would like to send data with a request, for example a **`POST`** request, we can add the data to a dictionary and send it with our **`POST`** as the data parameter:

```
data = {'key': 'value',
        'key2': 'value2'}
r = requests.put('https://httpbin.org/put', data=data)
```

After our request, we can access the content in our response. To get our content in JSON, we can call the **`json()`** method of **`requests`**:

```
r = requests.get('https://api.github.com/events')
r.json()
```

From here, we can access the data as you would using a normal Python dictionary.

Take Note

To learn more about `requests` you can have a look at the **official documentation**.

Reddit JSON Endpoint

For our third-party API example, we will be using Reddit's public JSON endpoints to fetch posts from a subreddit. Unlike APIs that require developer accounts and authentication keys, Reddit's JSON feeds are freely accessible and return data in a format we can easily parse with Python.

We will create our API calls in a separate file; this enables us to simply import the necessary functions within our **`views.py`** to implement the features.

Save the following helper function in `functions/reddit.py`:

```
import requests

def get_reddit_posts(subreddit="python"):
    url = f"https://www.reddit.com/r/{subreddit}/.json"
    headers = {"User-Agent": "DjangoEcommerceApp/1.0"}  # Reddit
requires a User-Agent

    response = requests.get(url, headers=headers)

    if response.status_code != 200:
        return {"error": f"Failed to fetch data:
{response.status_code}"}

    data = response.json()
    posts = []

    for item in data["data"]["children"]:
        post = item["data"]
        posts.append({
            "title": post["title"],
```

```
            "author": post["author"],
            "url": post["url"]
        })

    return posts
```

At the start of our program, we don't need to go through a complex authentication process with keys and secrets. Reddit's public JSON endpoints are open and can be accessed directly with a simple GET request, as long as we include a User-Agent header. The `User-Agent` header tells Reddit who is making the call and prevents our request from being blocked.

Instead of setting up an authentication function, we simply define a helper function that builds the request URL, sends the GET request, and processes the JSON response. For example, in `functions/reddit.py` we created:

```
import requests

def get_reddit_posts(subreddit="python"):
    # Build the URL for the subreddit's JSON feed
    url = f"https://www.reddit.com/r/{subreddit}/.json"
    headers = {"User-Agent": "DjangoEcommerceApp/1.0"}  # Required by
Reddit

    # Make the request
    response = requests.get(url, headers=headers)

    # Handle errors
    if response.status_code != 200:
        print("Failed to fetch data.")
        return None

    # Parse the JSON response
    data = response.json()
    posts = []

    # Extract useful fields from each post
    for item in data["data"]["children"]:
        post = item["data"]
        posts.append({
```

```
            "title": post["title"],
            "author": post["author"],
            "url": post["url"]
        })

    return posts
```

Importing and Using Our Reddit Function

Now that we have created our helper function in **`functions/reddit.py`**, the next step is to use it inside our Django project. To do this, we import the function into our **`views.py`** file and call it when we want to display posts from Reddit.

This step shows how we connect our API call to Django's view system. Instead of just printing results in the console, we're now preparing them to be shown in the browser.

In **`views.py:`**

```
from django.shortcuts import render
from .functions.reddit import get_reddit_posts

def reddit_feed(request):
    # Call our helper function to fetch posts
    posts = get_reddit_posts("python")

    # Pass the posts into the template
    return render(request, "reddit_feed.html", {"posts": posts})
```

- We **import** the **`get_reddit_posts`** function from our helper file.
- Inside the view, we **call the function** with the name of the subreddit we want (in this case, **`"python"`**).
- The function returns a list of posts, which we then **send to the template** so they can be displayed on the page.

Displaying Reddit Posts in a View

Reddit's public JSON endpoints can be accessed directly. This means we don't need to go through a PIN or verifier step, we can simply call our helper function and use the data it returns.

Here's how we can bring the posts into a Django view:

In `views.py`:

```
from django.shortcuts import render
from .functions.reddit import get_reddit_posts

def reddit_feed(request):
    # Fetch posts from the "python" subreddit
    posts = get_reddit_posts("python")

    # Pass the posts into the template
    return render(request, "reddit_feed.html", {"posts": posts})
```

- We `import` the helper function we created earlier.
- Inside the view, we **call the function** with the name of the subreddit we want.
- The function returns a list of posts, which we then **send to the template** so they can be displayed in the browser.

Passing Data Into Django

With Reddit's JSON endpoints, we don't need to request access tokens or create special session objects. Instead, our focus is on taking the data we've fetched and making it available inside Django. This is how we connect the API call to our web application.

In `views.py`:

```
from django.shortcuts import render
from .functions.reddit import get_reddit_posts

def reddit_feed(request):
    # Fetch posts from the "python" subreddit
    posts = get_reddit_posts("python")
```

```
    # Pass the posts into the template
    return render(request, "reddit_feed.html", {"posts": posts})
```

- We import the helper function from `functions/reddit.py`.
- Inside the view, we call the function with the name of the subreddit we want.
- The function returns a list of posts, which we then pass into the template.

Using the Data in Our Application

With Reddit's JSON endpoints, we don't need to create a final OAuth session or manage access tokens. Once our helper function has fetched the posts, we can immediately use that data inside Django. The important part is making sure the data flows from the API call into our view, and then into the template.

In `views.py`:

```
from django.shortcuts import render
from .functions.reddit import get_reddit_posts

def reddit_feed(request):
    # Fetch posts from the "python" subreddit
    posts = get_reddit_posts("python")

    # Pass the posts into the template
    return render(request, "reddit_feed.html", {"posts": posts})
```

Here, the `reddit_feed` view acts as the bridge:

- It calls our helper function to get the posts.
- It stores those posts in a variable.
- It passes them into the template so they can be displayed on the page.

Unlike APIs that require tokens and repeated authentication, Reddit's JSON feed only requires a simple GET request with a User-Agent header. This means we can focus on how to use the data in our application, rather than managing sessions.

Creating the Template to Display Posts

Now that our view is passing Reddit data into the template, the final step is to show those posts on the page. We'll create a new template file called **reddit_feed.html**inside your templates folder.

In **reddit_feed.html**:

```
<!DOCTYPE html>
<html>
<head>
    <title>Reddit Feed</title>
</head>
<body>
    <h1>Posts from Reddit</h1>
    <ul>
        {% for post in posts %}
            <li>
                <strong>{{ post.title }}</strong><br>
                by {{ post.author }}<br>
                <a href="{{ post.url }}" target="_blank">View Post</a>
            </li>
        {% endfor %}
    </ul>
</body>
</html>
```

In the template, we loop through the list of posts that was passed from the view. For each post, we display its title, the author who submitted it, and a clickable link that takes the user to the original Reddit post. To achieve this, the template uses Django's template tags, such as **{% for %}** to iterate over the list and **{{ }}** to insert the values dynamically. This completes the flow of our application: the helper function in **functions/reddit.py** fetches posts from Reddit, the view in **views.py** calls the helper and passes the data to the template, and finally the template **reddit_feed.html** displays the posts in the browser.

Adding the URLconf

To make our new view accessible in the browser, we need to add a route in `urls.py`. This connects a URL path to the `reddit_feed` view we created.

In `urls.py`:

```
from django.urls import path
from . import views

urlpatterns = [
    path("reddit/", views.reddit_feed, name="reddit_feed"),
]
```

- We import `path` from Django's URL library and our `views` module.
- We add a new entry to `urlpatterns` that maps the URL `/reddit/` to the `reddit_feed` view.
- The `name="reddit_feed"` part gives this route a name, which can be useful when linking to it in templates.

Now, when you start your Django server and visit http://localhost:8000/reddit/, the application will call the `reddit_feed` view, fetch posts from Reddit, and display them using the `reddit_feed.html` template.

Practical task 1

In this task, you will extend the functionality of your eCommerce project to allow users to interact with your service via a RESTful API.

Before adding the functionality to your code, you need to plan a few things:

- Think about your models and how you would like to serialise them and have your resources represented (JSON/XML).
- Carefully think about your endpoints as this is how users will access your resources. You can see here for a good set of **best practices**.
- Create a sequence diagram of the interactions a user will have with your API.

Follow these steps:

1. Allow vendors to create new stores and add products to their stores using your web API. They should also be able to retrieve reviews. (Remember to perform authentication before allowing a user to edit resources.)
2. Buyers and vendors should be allowed to retrieve the stores listed under each vendor and the products of each store.

Practical task 2

Now that you have our own API for third-party users, let's incorporate a third-party API into your project.

Follow these steps:

1. Decide which subreddit you would like to fetch posts from (for example, `python`, `django`, or any topic of interest).
2. Create a helper function in a new file called `functions/reddit.py`.
    a. This function should:
        i. build the request URL for the subreddit,
        ii. send a GET request with a `User-Agent` header,

iii. and parse the JSON response into a list of posts.

3. Import your helper function into `views.py` and create a new view (e.g., `reddit_feed`) that calls the function and passes the posts into a template.
4. Create a template called `reddit_feed.html` that loops through the posts and displays the
    a. title,
    b. author,
    c. and a clickable link to the original Reddit post.
5. Add a URL pattern in `urls.py` that maps a path (e.g., `/reddit/`) to your new view so that users can access the Reddit feed in their browser.
6. Test your integration by running the server and visiting the `/reddit/` endpoint.
    a. Confirm that posts from your chosen subreddit are displayed correctly.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.

Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.